

Capstone Project and Test Summary

(PLAYWRIGHT AUTOMATION TESTING)

-Keerthana R

B.E Computer Science &Engineering
SDET Playwright

Overview

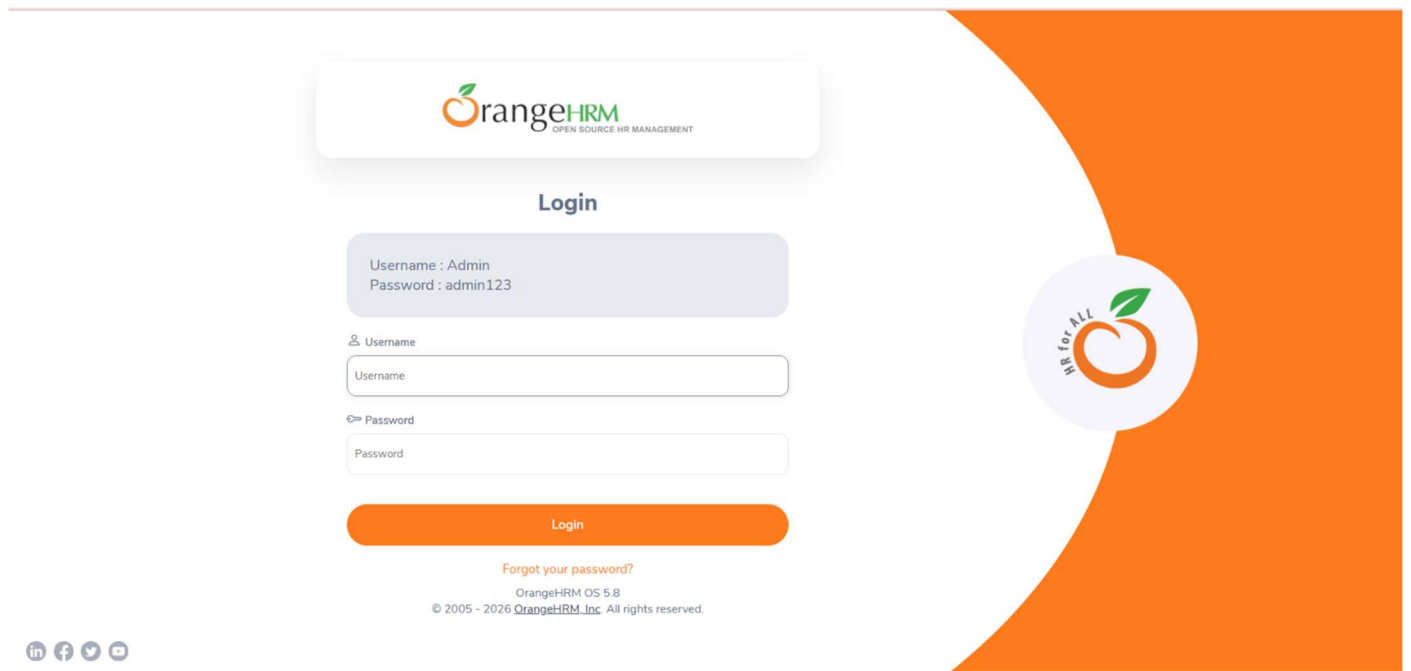
Project Overview

The Application Under Test (AUT) is OrangeHRM, a web-based Human Resource Management System designed to streamline and manage various HR operations within an organization. The platform provides comprehensive functionalities such as employee information management, recruitment, leave management, performance tracking, time management, and user administration through a centralized and user-friendly interface.

This project focuses on ensuring the quality, reliability, and performance of the OrangeHRM application by validating its core business processes and system functionalities. The testing scope covers critical modules including Login, Dashboard, PIM (Personnel Information Management), Recruitment, Leave Management, Time Tracking, and Employee Administration.

The automation framework is developed using Playwright with JavaScript, following the Page Object Model (POM) design pattern to improve test maintainability, scalability, and reusability. The framework also integrates GitHub Actions for continuous integration and continuous deployment (CI/CD), along with Allure Reports for detailed test execution reporting and analysis.

The primary objective of this project is to verify that OrangeHRM meets functional requirements, maintains data integrity, ensures system stability, and delivers a seamless user experience across all HR management modules.



Goals

The primary quality objectives for this engagement are defined as follows:

1 Business Objectives

- Employee Data Integrity: Ensure 100% accuracy in employee information management, user administration, leave records, timesheets, and performance data.
- Operational Efficiency: Validate seamless workflows across HR processes such as employee onboarding, leave requests, recruitment management, and time tracking.
- Data Security: Verify that sensitive employee information, login credentials, and administrative functions are protected through proper authentication and authorization mechanisms.
- System Reliability: Ensure critical HR operations function consistently without data loss or workflow interruptions.

2 Quality Engineering Objectives

- Automation Coverage: Achieve high automation coverage for repetitive functional and regression test scenarios using Playwright and JavaScript.

- Defect Containment: Identify and resolve critical and high-severity defects before production deployment, with special focus on employee management, leave processing, and user administration modules.
- Data Validation: Ensure UI actions correctly update application records and maintain data consistency across the system.

Specifications

In-Scope Functional Module

Testing will cover the following core functional areas:

1.Authentication and User Management

- Validate successful and unsuccessful login scenarios.
- Verify logout functionality and session management.
- Validate role-based access controls and user permissions.

2.Admin Module

- Verify user creation, modification, search, and deletion.
- Validate user roles and status management.

3.PIM (Personnel Information Management)

- Verify employee creation, update, search, and deletion.
- Validate employee record accuracy and data persistence.

4.Leave Management

- Validate leave application, approval, rejection, and leave balance calculations.
- Verify leave history and leave status updates.

5.Time Management

- Verify timesheet creation, submission, editing, and validation.
- Validate recorded work hours and time entries.

6.Recruitment Management

- Verify candidate status tracking and vacancy management.
- Validate recruitment workflow operations.

7.Dashboard and Profile Management

- Verify dashboard widgets and role-based visibility.
- Validate profile updates and password change functionality.

8.Expenses and Performance Management

- Verify employee expense claims, expense categories, and claim submission.
- Validate employee performance tracking, reviews, and performance configuration.

Milestone 1 - Analysis and Planning

1.1 Introduction

The objective of this phase is to analyze the business requirements of the OrangeHRM application and define a comprehensive testing strategy. This phase ensures that all functional and non-functional requirements are testable and that potential risks are identified early in the Software Testing Life Cycle (STLC).

1.2 Requirement Analysis

The following Functional Requirements (FR) and Non-Functional Requirements (NFR) have been identified based on the OrangeHRM business processes and project scope.

1.2.1 Functional Requirements (FR)

Module	Requirement Description	Priority
Authentication	Users must be able to log in with valid credentials and access authorized modules. Invalid credentials must be rejected.	Critical
Admin	Administrators must be able to add, search, edit, and delete user accounts while managing user roles and permissions.	High
PIM	Users must be able to create, update, search, and manage employee records.	Critical
Leave	Employees must be able to apply for leave and authorized users must approve or reject leave requests.	High
Time	Employees must be able to create, submit, edit, and manage timesheets.	High
Recruitment	HR users must be able to manage vacancies and track candidate status.	Medium

Module	Requirement Description	Priority
Profile	Users must be able to view and update personal information and change passwords.	Medium
Dashboard	The system must display appropriate widgets and information based on user roles.	Medium
Expenses	Employees must be able to submit and manage expense claims.	Medium
Performance	Managers must be able to track employee performance and conduct performance reviews.	Medium

1.2.2 Non-Functional Requirements (NFR)

- Data Integrity: Employee records, leave requests, timesheets, and recruitment data must remain accurate and consistent throughout the application.
- Browser Compatibility: The application should function correctly across supported browsers tested through Playwright execution environments.
- Performance: Core HR workflows should execute efficiently without significant delays.
- Security: Authentication and role-based access controls must prevent unauthorized access to sensitive HR data.

1.3 Test Strategy

The testing strategy focuses on end-to-end automation testing using Playwright and JavaScript.

1.3.1 Automation Testing Scope

Automation testing will be used for:

- Authentication Testing: Login, logout, session validation, and credential verification.
- Admin Module Testing: User creation, search, modification, deletion, and role validation.
- PIM Testing: Employee creation, update, search, and deletion.
- Leave Management Testing: Leave application, approval, rejection, and leave balance validation.
- Time Module Testing: Timesheet creation, submission, editing, and validation.
- Recruitment Testing: Vacancy management and candidate status tracking.
- Profile Testing: Personal information updates and password changes.

- Expenses and Performance Testing: Expense claims, performance tracking, and employee reviews.
- Regression Testing: Validation of existing functionalities after application changes.

1.4 Requirements Traceability Matrix (RTM) Structure

To ensure complete test coverage, an RTM will be maintained linking every requirement to corresponding test cases.

Requirement Summary	Test Scenario
Authentication	Verify successful login with valid credentials
Authentication	Verify error message for invalid credentials
Admin Management	Verify successful user creation
PIM Management	Verify employee record creation
Leave Management	Verify leave application and approval workflow
Time Management	Verify timesheet submission
Recruitment	Verify candidate status tracking
Profile Management	Verify password change functionality
Expenses Management	Verify expense claim submission
Performance Management	Verify employee performance review workflow

Milestone 2 - Automation Framework Setup and Test Script Development

2.1 Framework Setup and Configuration

The Playwright automation framework was configured using JavaScript and Node.js. The project was organized using the Page Object Model (POM) design pattern to improve code reusability, maintainability, and scalability.

Playwright Installation

The Playwright automation framework was installed using Node.js and npm. Required dependencies and browser binaries were successfully configured to enable automated test execution across supported browsers.

```
Inside that directory, you can run several commands:

npx playwright test
  Runs the end-to-end tests.

npx playwright test --ui
  Starts the interactive UI mode.

npx playwright test --project=chromium
  Runs the tests only on Desktop Chrome.

npx playwright test example
  Runs the tests in a specific file.

npx playwright test --debug
  Runs the tests in debug mode.

npx playwright codegen
  Auto generate tests with Codegen.

We suggest that you begin by typing:

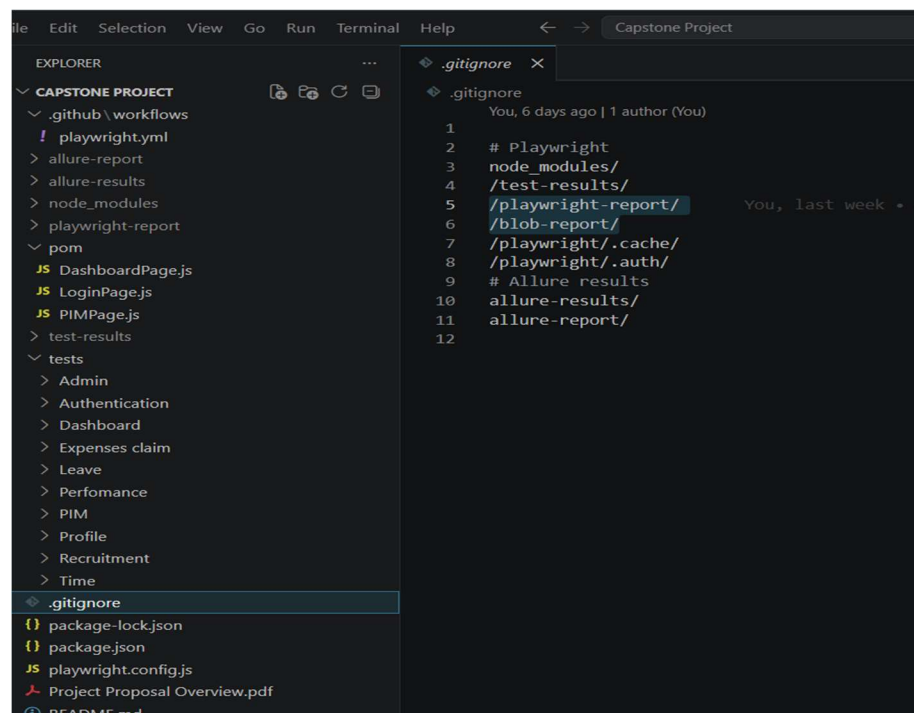
npx playwright test

And check out the following files:
- .\tests\example.spec.ts - Example end-to-end test
- .\tests-examples\demo-todo-app.spec.ts - Demo Todo App end-to-end tests
- .\playwright.config.ts - Playwright Test configuration

Visit https://playwright.dev/docs/intro for more information. ✨

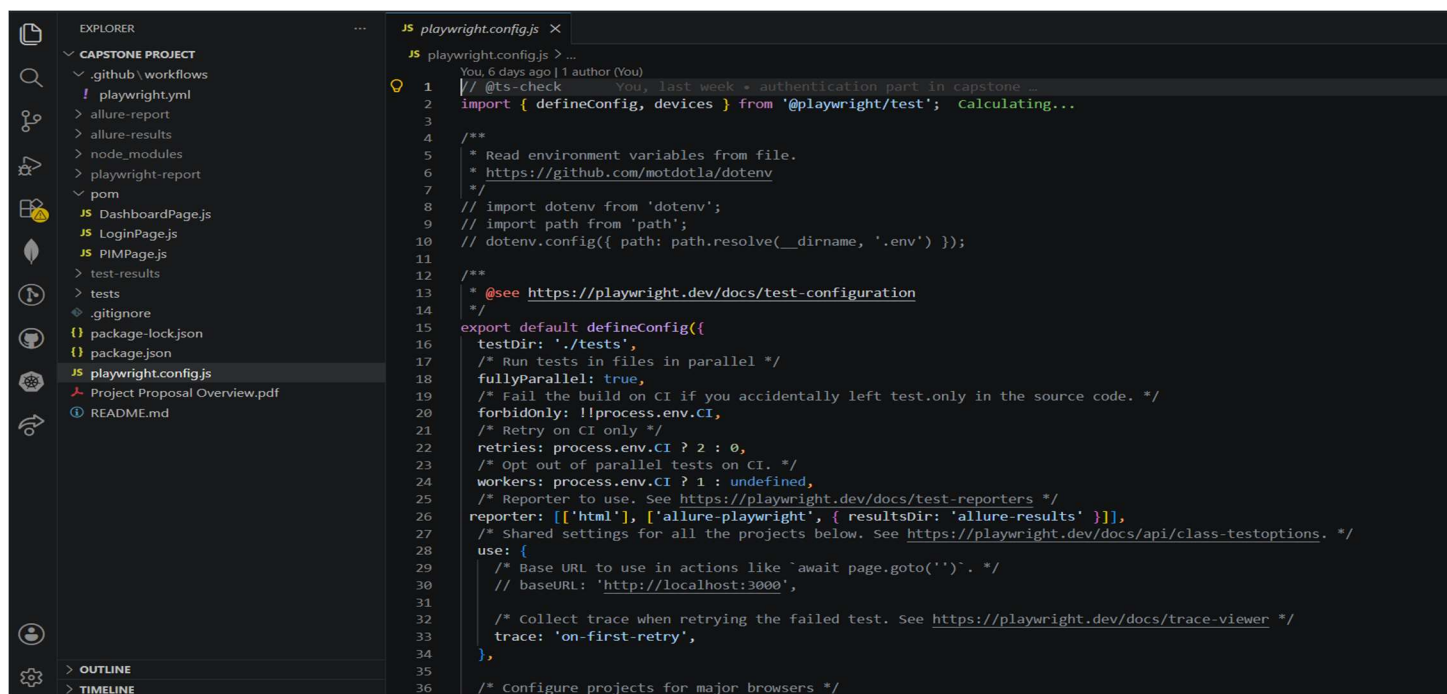
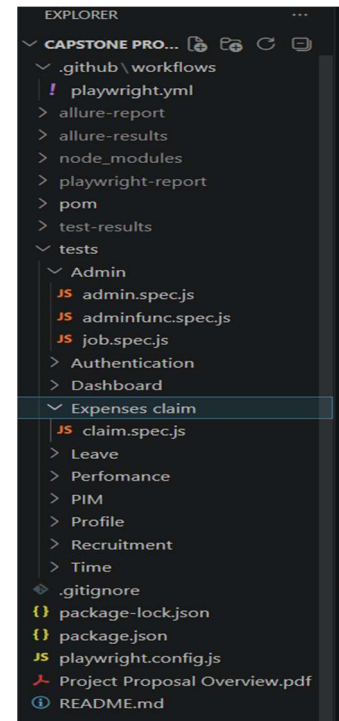
Happy hacking! 🍴
```

2.1 Playwright Installation



The framework includes:

- Page Object Model (POM) classes
- Test scripts for OrangeHRM modules
- Playwright configuration file
- Playwright HTML reports
- Allure reporting integration
- GitHub Actions workflow for CI/CD execution

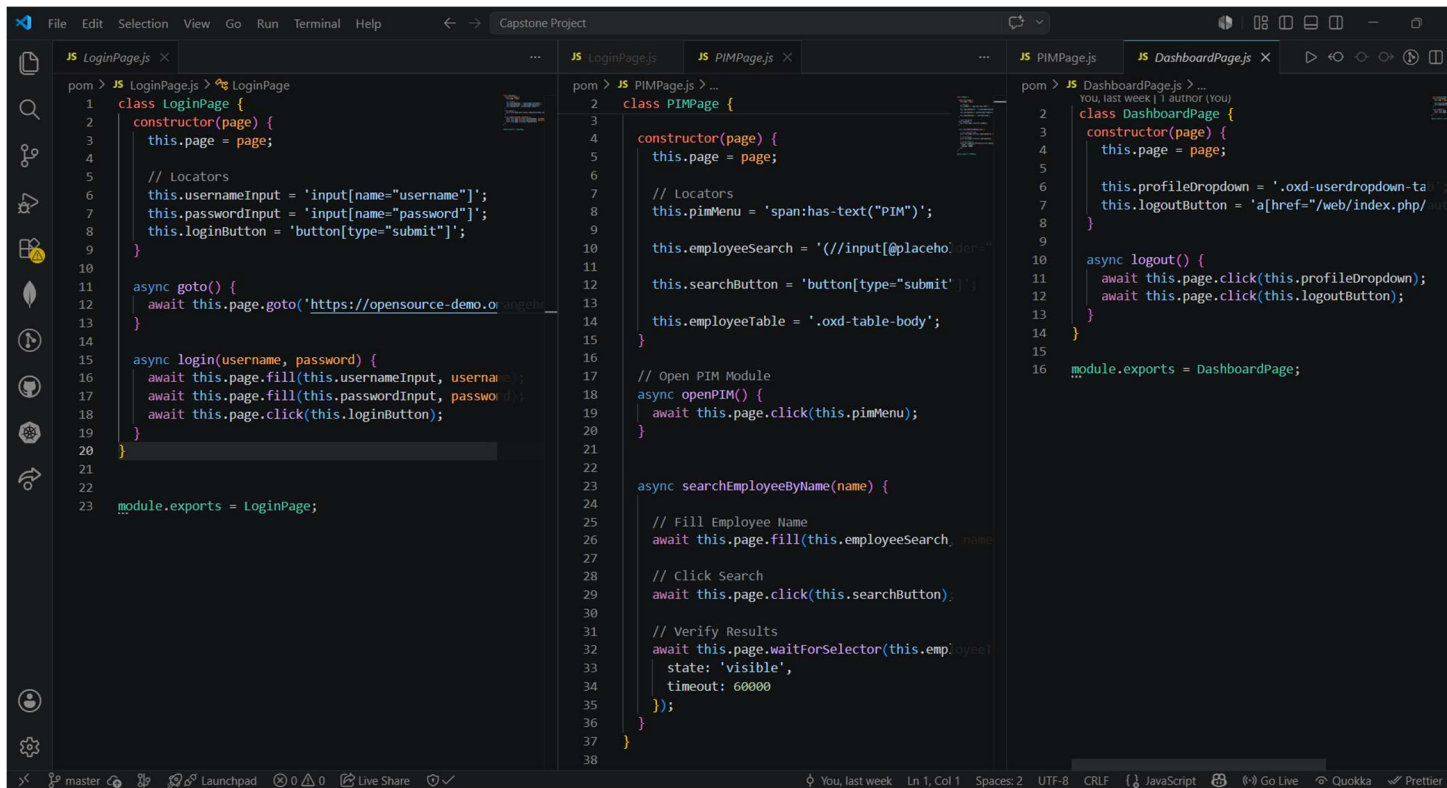


2.1 Playwright configuration file

2.2 Page Object Model (POM) Implementation

The Page Object Model design pattern was implemented to separate page elements and actions from test scripts. This approach improves maintainability and reduces code duplication.

Page classes were created for different OrangeHRM modules such as Login, Dashboard, and PIM.



```
JS LoginPage.js
pom > JS LoginPage.js > LoginPage
1 class LoginPage {
2   constructor(page) {
3     this.page = page;
4
5     // Locators
6     this.usernameInput = 'input[name="username"]';
7     this.passwordInput = 'input[name="password"]';
8     this.loginButton = 'button[type="submit"]';
9   }
10
11   async goto() {
12     await this.page.goto('https://opensource-demo.orangehrmlive.com/');
13   }
14
15   async login(username, password) {
16     await this.page.fill(this.usernameInput, username);
17     await this.page.fill(this.passwordInput, password);
18     await this.page.click(this.loginButton);
19   }
20 }
21
22
23 module.exports = LoginPage;

JS PIMPage.js
pom > JS PIMPage.js > PIMPage
1 class PIMPage {
2   constructor(page) {
3     this.page = page;
4
5     // Locators
6     this.pimMenu = 'span:has-text("PIM")';
7
8     this.employeeSearch = 'input[placeholder="Search Employee Name"]';
9     this.searchButton = 'button[type="submit"]';
10    this.employeeTable = '.oxd-table-body';
11  }
12
13  // Open PIM Module
14  async openPIM() {
15    await this.page.click(this.pimMenu);
16  }
17
18  async searchEmployeeByName(name) {
19    // Fill Employee Name
20    await this.page.fill(this.employeeSearch, name);
21
22    // Click Search
23    await this.page.click(this.searchButton);
24
25    // Verify Results
26    await this.page.waitForSelector(this.employeeTable, {
27      state: 'visible',
28      timeout: 60000
29    });
30  }
31 }
32
33 module.exports = PIMPage;

JS DashboardPage.js
pom > JS DashboardPage.js > DashboardPage
1 class DashboardPage {
2   constructor(page) {
3     this.page = page;
4
5     this.profileDropdown = '.oxd-userdropdown-toggle';
6     this.logoutButton = 'a[href="/web/index.php/logout"]';
7   }
8
9   async logout() {
10    await this.page.click(this.profileDropdown);
11    await this.page.click(this.logoutButton);
12  }
13 }
14
15
16 module.exports = DashboardPage;
```

2.3 Test Script Development

Automation test scripts were developed to validate major OrangeHRM functionalities including:

- Authentication Module
- Admin Module
- PIM Module
- Leave Module
- Time Module

- Dashboard Module
- Profile Module
- Recruitment Module
- Expenses Module
- Performance Module

These scripts validate various positive and negative test scenarios to ensure application functionality and stability.

```

1  const { test, expect } = require('@playwright/test');
2  const LoginPage = require('../../pom/LoginPage');
3
4  test.describe('Admin Tests', () => {
5
6      let loginPage;
7
8      test.beforeEach(async ({ page }) => {
9          page.setDefaultTimeout(60000);
10         loginPage = new LoginPage(page);
11         await loginPage.goto();
12         await loginPage.login('Admin', 'admin123');
13         await page.getByRole('link', { name: 'Admin' }).click();
14     });
15
16     test('Open Admin module', async ({ page }) => {
17         await page.getByRole('link', { name: 'Admin' }).click();
18     });
19
20     test('Search for user', async ({ page }) => {
21         await page.getByRole('textbox').nth(1).fill('Mandaa user');
22         await page.locator('.oxd-icon.bi-caret-down-fill.oxd-select-text--arrow').first().click();
23         await page.getByRole('button', { name: 'Search' }).click();
24     });
25
26     test('Add user', async ({ page }) => {
27         await page.getByRole('button', { name: 'Add' }).click();
28     });
29
30     test('Open Nationalities page', async ({ page }) => {
31         await page.getByRole('link', { name: 'Nationalities' }).click();
32     });
33
34     test('to know about work shifts', async ({ page }) => {
35         await page.getByText('Job').click();
36         await page.getByRole('menuItem', { name: 'Work Shifts' }).click();
37     });
38
39 });

```

2.4 Test Execution

The developed automation scripts were executed using the Playwright Test Runner across configured browser environments. Test execution results, screenshots, logs, and reports were generated to validate application functionality and identify failures efficiently.

```

PS C:\Users\thili\Desktop\Capstone Project> npx playwright test --project=chromium

Running 125 tests using 4 workers
1) [chromium] > tests\PIM\employeevalidation.spec.js:23:3 > PIM - Employee Validation Tests > Verify Add Employee page loads correctly

Test timeout of 30000ms exceeded.

Error: page.click: Test timeout of 30000ms exceeded.
Call log:
  - waiting for locator('a:has-text("Add Employees")')

23 |   test('Verify Add Employee page loads correctly', async ({ page
    |   }) => {
24 |
    |   > 25 |     await page.click('a:has-text("Add Employees")');
    |         ^
26 |
27 |     await expect(
28 |       page.getByRole('heading', { name: 'Add Employee' })
    |       at C:\Users\thili\Desktop\Capstone Project\tests\PIM\employeevalidation.spec.js:25:16

Error Context: test-results\PIM-employeevalidation-PIM-072f9-ployee-page-loads-correctly-chromium\error-context.md

1 failed
[chromium] > tests\PIM\employeevalidation.spec.js:23:3 > PIM - Employee Validation Tests > Verify Add Employee page loads correctly
124 passed (6.5m)

```

The screenshot shows the VS Code interface with the 'error-context.md' file open in the editor. The file contains the following content:

```

1 # Instructions
2
3 - Following Playwright test failed.
4 - Explain why, be concise, respect Playwright best practices.
5 - Provide a snippet of code with the fix, if possible.
6
7 # Test info
8
9 - Name: PIM\employeevalidation.spec.js >> PIM - Employee Validation Tests >> Verify Add Employee page loads correctly
10 - Location: tests\PIM\employeevalidation.spec.js:23:3
11
12 # Error details
13
14 ...
15 Test timeout of 30000ms exceeded.
16 ...
17
18 ...
19 Error: page.click: Test timeout of 30000ms exceeded.
20 Call log:
21   - waiting for locator('a:has-text("Add Employees")')
22 ...
23
24 # Page snapshot
25
26 ...yaml
27
28 - generic [ref=e3]:
29   - generic:
30     - complementary [ref=e4]:
31       - navigation "Sidepanel" [ref=e5]:
32         - generic [ref=e6]:
33           - link "Client brand banner" [ref=e7] [cursor=pointer]:
34             - /url: https://www.orangehrm.com/
35             - img "client brand banner" [ref=e9]
36             - text:
37               - generic [ref=e10]:

```

The left sidebar shows the project structure, including files like 'playwright.yml', 'allure-report', 'node_modules', 'playwright-report', 'pom', 'DashboardPage.js', 'LoginPage.js', 'PIMPage.js', 'test-results', 'PIM-employeevalidation-PIM-072f9-ployee...', 'error-context.md', 'last-run.json', 'tests', 'Admin', 'Authentication', 'Dashboard', 'Expenses claim', 'Leave', 'Performance', 'PIM', 'Profile', 'Recruitment', 'Time', '.gitignore', 'package-lock.json', 'package.json', 'playwright.config.js', 'Project Proposal Overview.pdf', 'OUTLINE', and 'TIMELINE'.

2.5 Test Reporting

Playwright HTML Reports and Allure Reports were integrated into the framework to provide detailed test execution summaries, pass/fail statistics, and failure analysis.

2.6 Continuous Integration Setup

GitHub Actions was configured to automatically trigger Playwright test execution on code commits and pull requests. Test reports and execution artifacts were generated as part of the CI/CD workflow to support continuous quality validation.

2.7 Outcome

The automation framework was successfully established, and automated test scripts were developed for multiple OrangeHRM modules. Reporting and CI/CD integration were also configured to support continuous testing and efficient result analysis.

Milestone 3 - Automated Test Execution and Validation

3.1 Test Execution Strategy

After developing the automation framework and test scripts, automated test cases were executed using Playwright Test Runner. The objective was to validate the functionality of the OrangeHRM application and verify that all automated workflows performed as expected.

3.2 Test Coverage

Automated test cases were executed for the following modules:

- Authentication Module
- Admin Module
- PIM Module
- Leave Module
- Time Module
- Dashboard Module
- Profile Module
- Recruitment Module

- Expenses Module
- Performance Module

3.3 Test Design Methodology

The automated test cases were designed based on business requirements and user workflows within the OrangeHRM application.

The design followed these principles:

1. Functional Validation
 - Verify that application features behave according to requirements.
2. Positive and Negative Testing
 - Validate successful operations and error handling scenarios.
3. Regression Testing
 - Ensure existing functionalities remain unaffected after changes.
4. Requirement Traceability
 - Maintain mapping between requirements and automated test cases.

3.4 Test Cases

Authentication Module

- Verify successful login with valid credentials
- Verify login failure with invalid credentials
- Verify empty username validation
- Verify empty password validation
- Verify logout functionality

Admin Module

- Verify add user
- Verify search user
- Verify edit user
- Verify delete user

PIM Module

- Verify add employee

- Verify search employee
- Verify update employee details
- Verify employee record validation

Leave Module

- Verify leave application
- Verify leave approval
- Verify leave rejection

Time Module

- Verify timesheet submission
- Verify timesheet update

Recruitment Module

- Verify candidate status validation
- Verify vacancy information

Profile Module

- Verify profile update
- Verify password change

Expenses Module

- Verify expense claim submission

Performance Module

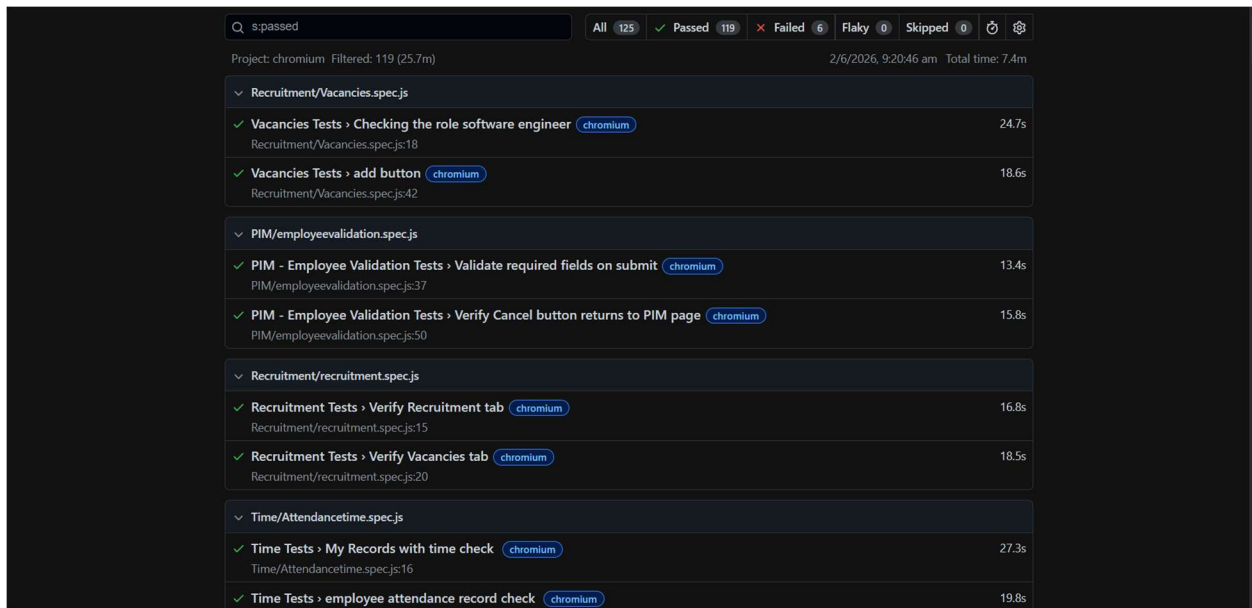
- Verify employee tracker validation
- Verify performance review validation

Milestone 4 - Reporting, Result Analysis and CI/CD Integration

4.1 Playwright Report Generation

After executing the automated test suite, Playwright's built-in reporting functionality was used to generate detailed execution reports. These reports provide information regarding test execution status, execution duration, pass/fail statistics, and failure details.

The Playwright report enabled efficient analysis of test results and helped identify any failed scenarios during execution.



The screenshot displays the Playwright HTML Report Dashboard. At the top, there is a search bar with 's:passed' and a summary bar showing 'All 125', 'Passed 119', 'Failed 6', 'Flaky 0', and 'Skipped 0'. Below this, the project name 'chromium' and a filter 'Filtered: 119 (25.7m)' are shown. The date and time '2/6/2026, 9:20:46 am' and total time 'Total time: 7.4m' are also present. The main content area lists test results for five modules: Recruitment/Vacancies.spec.js, PIM/employeevalidation.spec.js, Recruitment/recruitment.spec.js, and Time/Attendancetime.spec.js. Each module entry includes a green checkmark, the test name, the browser used (chromium), and the execution time.

Module	Test Name	Browser	Duration
Recruitment/Vacancies.spec.js	Vacancies Tests > Checking the role software engineer	chromium	24.7s
	Vacancies Tests > add button	chromium	18.6s
PIM/employeevalidation.spec.js	PIM - Employee Validation Tests > Validate required fields on submit	chromium	13.4s
	PIM - Employee Validation Tests > Verify Cancel button returns to PIM page	chromium	15.8s
Recruitment/recruitment.spec.js	Recruitment Tests > Verify Recruitment tab	chromium	16.8s
	Recruitment Tests > Verify Vacancies tab	chromium	18.5s
Time/Attendancetime.spec.js	Time Tests > My Records with time check	chromium	27.3s
	Time Tests > employee attendance record check	chromium	19.8s

4.1: Playwright HTML Report Dashboard

4.2 Test Result Analysis

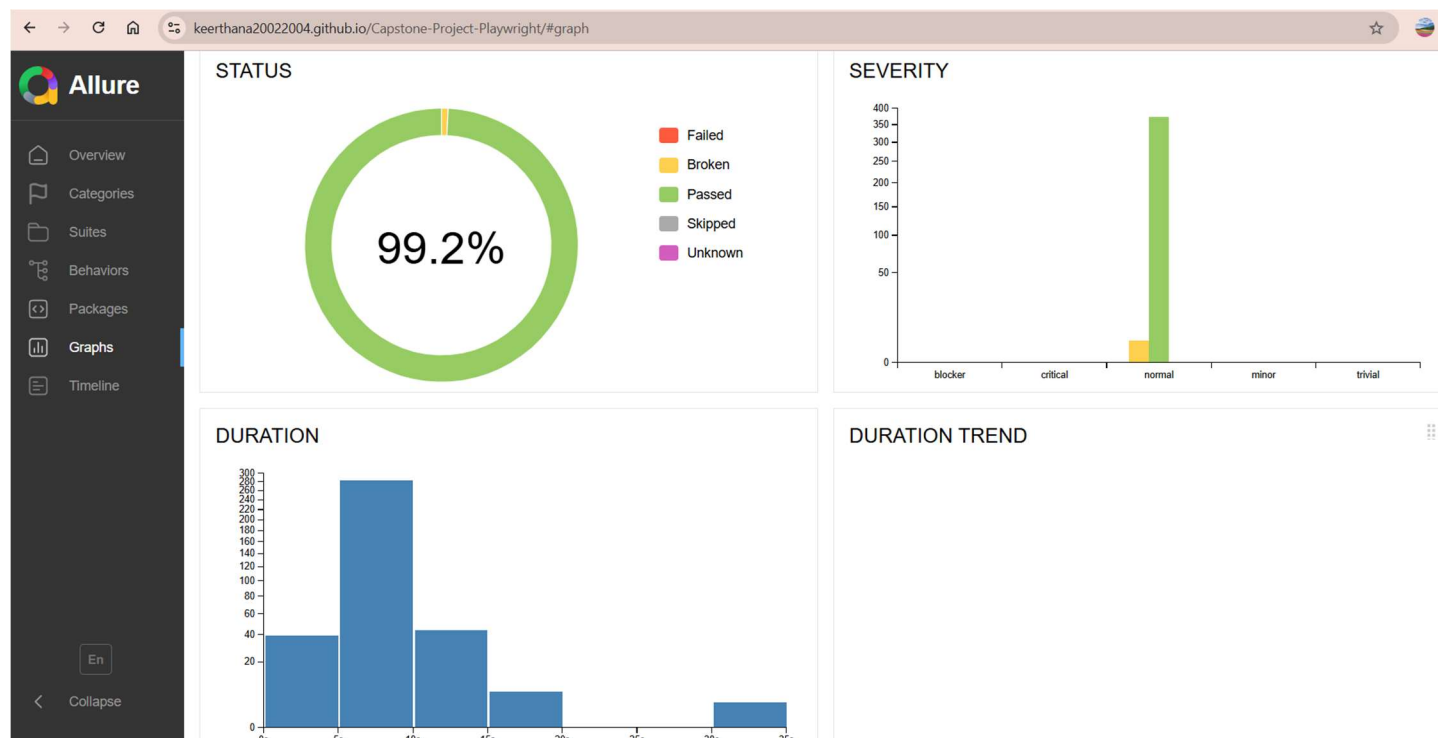
The generated reports were analyzed to verify the overall stability and functionality of the OrangeHRM application.

The analysis included:

- Total number of executed test cases
- Number of passed test cases
- Number of failed test cases
- Execution duration
- Module-wise test coverage
- Error and failure investigation

The successful execution of the automated test suite confirmed that the implemented OrangeHRM modules functioned according to the defined requirements.

Metric	Value
Total Test Cases	125
Passed	124
Failed	1
Skipped	0
Pass Percentage	99.2%



4.2 Test Execution Summary

4.3 Defect Tracking and Validation

Any issues identified during execution were analyzed and corrected. The affected test cases were re-executed to confirm successful resolution.

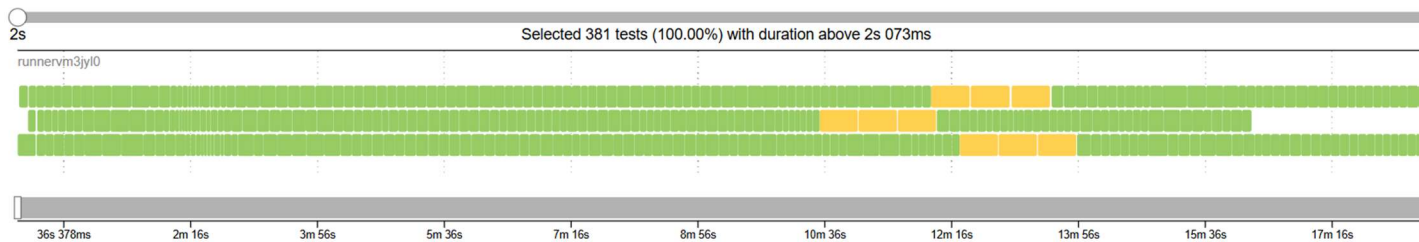
Regression testing was performed to ensure that fixes did not impact existing functionalities within the application.

4.4 Allure Report Integration

Allure Reports were integrated with the Playwright automation framework to provide advanced reporting capabilities and detailed test execution insights.

The generated reports include:

- Test execution summary
- Pass and fail statistics
- Execution timeline
- Detailed test information
- Failure screenshots and logs
- Historical execution trends



4.4 Execution timeline

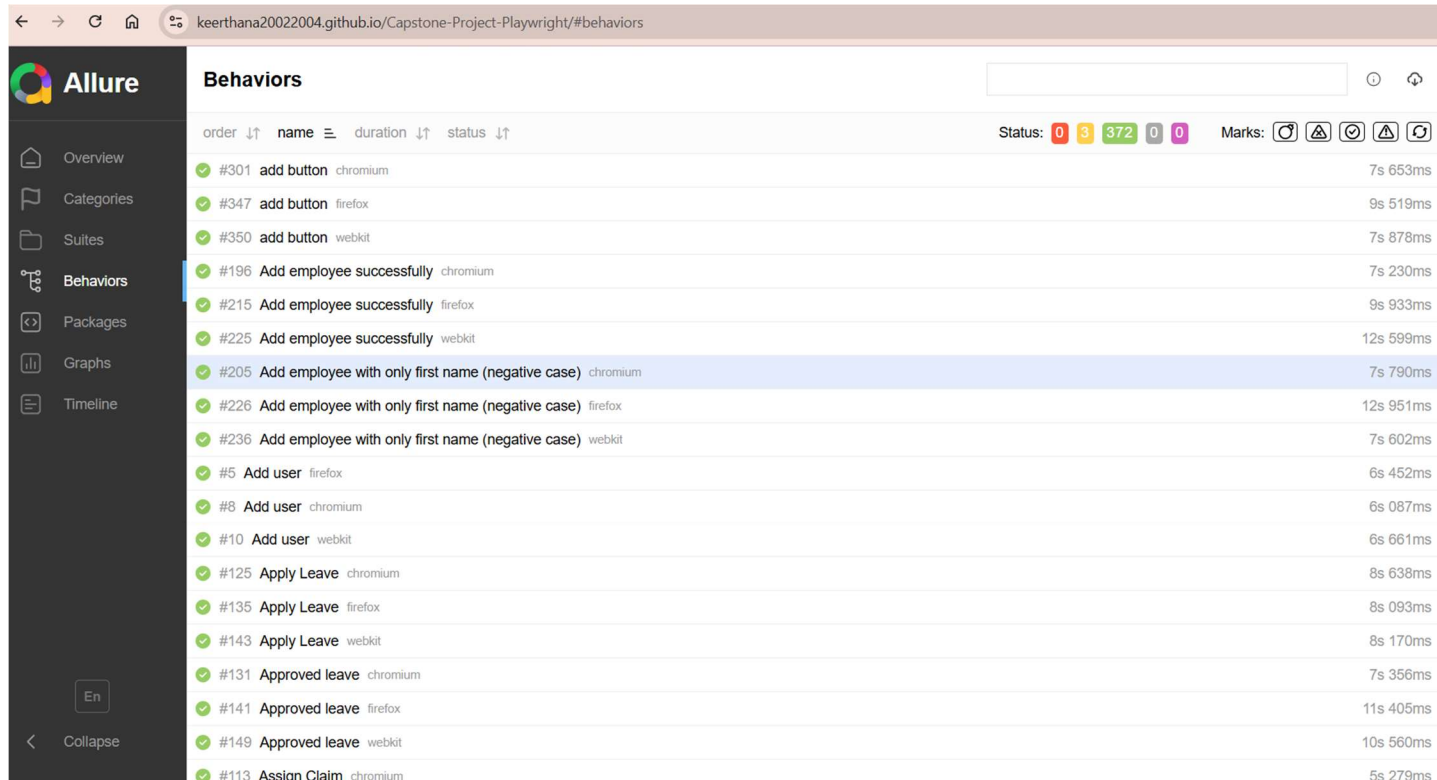
4.5 Module-wise Test Reporting

The Allure report provides detailed execution information for each OrangeHRM module.

Modules included in the automation suite:

- Authentication Module
- Admin Module
- PIM Module
- Leave Module
- Time Module
- Dashboard Module
- Profile Module
- Recruitment Module

- Expenses Module
- Performance Module



order	name	duration	status
#301	add button chromium	7s 653ms	Success
#347	add button firefox	9s 519ms	Success
#350	add button webkit	7s 878ms	Success
#196	Add employee successfully chromium	7s 230ms	Success
#215	Add employee successfully firefox	9s 933ms	Success
#225	Add employee successfully webkit	12s 599ms	Success
#205	Add employee with only first name (negative case) chromium	7s 790ms	Success
#226	Add employee with only first name (negative case) firefox	12s 951ms	Success
#236	Add employee with only first name (negative case) webkit	7s 602ms	Success
#5	Add user firefox	6s 452ms	Success
#8	Add user chromium	6s 087ms	Success
#10	Add user webkit	6s 661ms	Success
#125	Apply Leave chromium	8s 638ms	Success
#135	Apply Leave firefox	8s 093ms	Success
#143	Apply Leave webkit	8s 170ms	Success
#131	Approved leave chromium	7s 356ms	Success
#141	Approved leave firefox	11s 405ms	Success
#149	Approved leave webkit	10s 560ms	Success
#113	Assian Claim chromium	5s 279ms	Success

4.5: Module-wise Allure Results

4.6 Continuous Integration using GitHub Actions

GitHub Actions was configured to automate the execution of Playwright test cases whenever code changes were pushed to the repository.

The CI/CD workflow performs the following activities:

- Source code checkout
- Dependency installation
- Playwright test execution
- Report generation
- Result publication

This automation ensures continuous validation of application functionality and supports efficient regression testing.

Files

master

Go to file

.github/workflows

playwright.yml

pom

tests

.gitignore

Project Proposal Overview.pdf

README.md

package-lock.json

package.json

playwright.config.js

KEERTHANA20022004 capstone proj

Code Blame 104 lines (87 loc) · 2.37 KB

```
1  name: Playwright Tests & Allure Report
2  on:
3    push:
4      branches:
5        - "*"
6    pull_request:
7      branches: [ main, master ]
8    workflow_dispatch:
9
10 permissions:
11   contents: write
12   pages: write
13   id-token: write
14
15 jobs:
16   test:
17     timeout-minutes: 60
18     runs-on: ubuntu-latest
19     strategy:
20       fail-fast: false
21     matrix:
22       shard_index: [1, 2, 3]
23     env:
24       CI: true
25
26     steps:
27       - name: Checkout Repository
28         uses: actions/checkout@v4
29
30       - name: Setup Node.js
```

4.6: GitHub Actions Workflow File

KEERTHANA20022004 / Capstone-Project-Playwright

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

pages build and deployment #9

Summary

All jobs

build

report-build-status

deploy

Run details

Usage

Triggered via GitHub Pages 2 days ago

Status

Total duration

Artifacts

github-pages[bot] 67459f4 gh-pages Success 26s 1

pages-build-deployment

on: dynamic

build 9s

report-build-status 4s

deploy 8s

https://keerthana20022004.github.io/Caps...

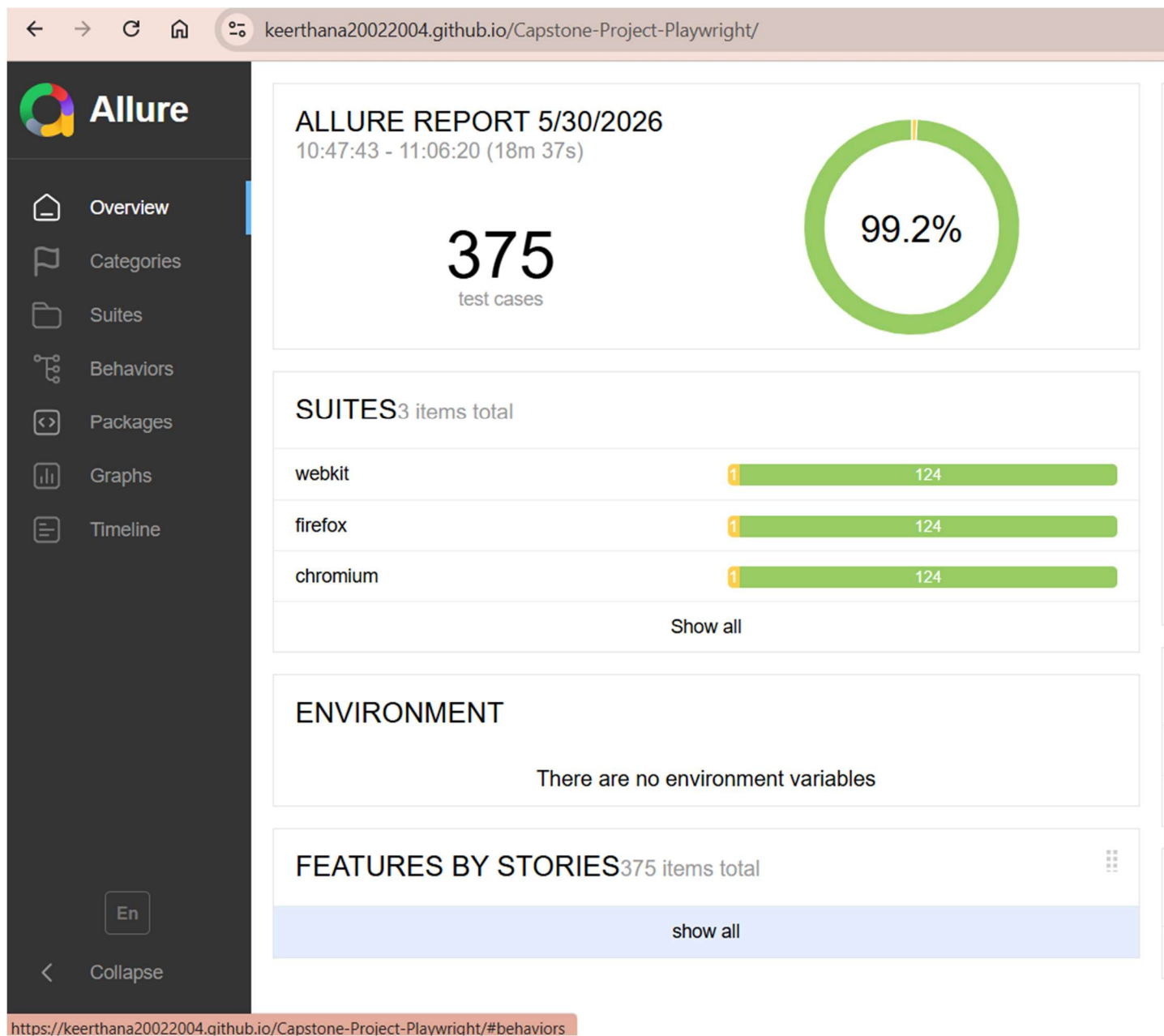
Annotations

4.7: Successful GitHub Actions Execution

4.7 Outcome

The integration of Playwright automation, Allure Reports, and GitHub Actions provided a scalable and maintainable test automation solution for OrangeHRM.

The framework supports automated regression testing, detailed reporting, and continuous quality validation, improving overall software quality and testing efficiency.



4.8: Final Allure Summary Report